

Intro

Problem: DFD relies on image derivatives which are noise sensitive, thus, DFD fails in photon-limited scenarios.

Solution: Capture pair of defocused images -> same structure but different boundary smoothness -> parameterize patches as Blurry Edges repr. -> analytical depth eqn.

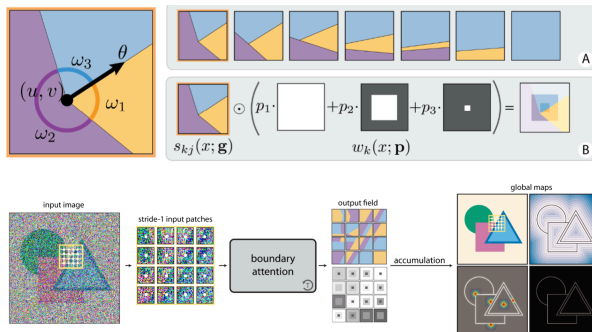
BlurryEdges Representation

Inspiration

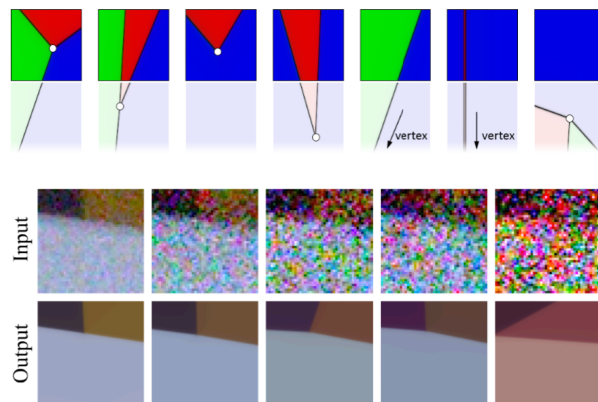
Inspired by "field of junction" boundary extraction technique

1. DNN turns noisy image patches into clean geometry for boundary extraction

<https://arxiv.org/pdf/2401.00935> - Boundary Attention: Learning curves, corners, junctions and grouping



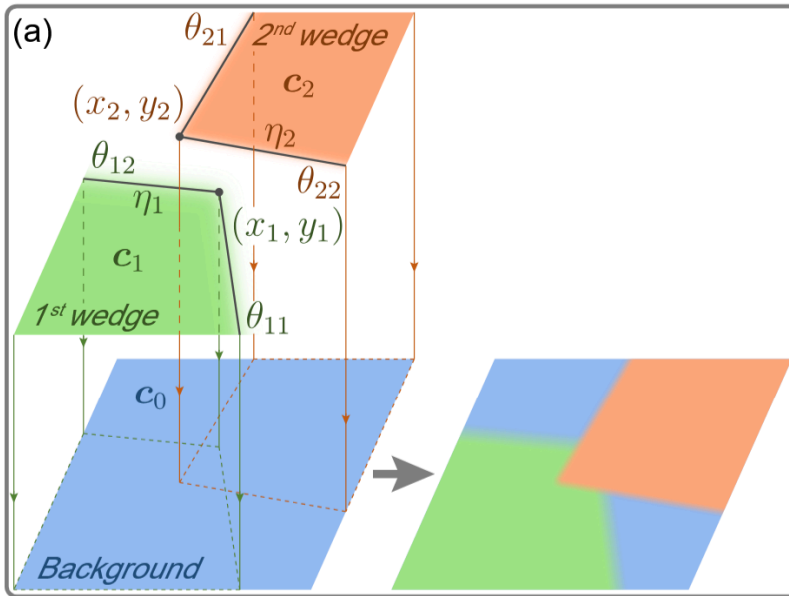
<https://arxiv.org/pdf/2011.13866> - Field of Junctions: Extracting Boundary Structure at Low SNR



Generalized FoJ to BlurryEdges

Generalized to include boundary smoothness and overlapping shapes

Representation



Each image patch is represented as the alpha clipping of l vertically-stacked, constant color wedges with smooth boundaries is parameterized by:

$$\Psi = (\{p_i, \theta_i, c_i, \eta_i, i = 1, 2, \dots, l\}, c_0)$$

Where:

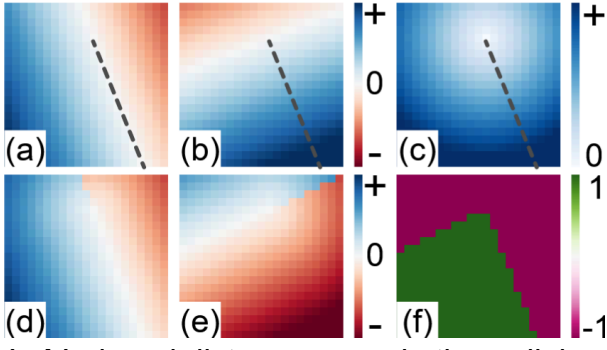
1. The tuple $(p_i, \theta_i, c_i, \eta_i)$ parameterizes the i th wedge
 1. $p_i = (x_i, y_i)$ is the vertex
 2. $\theta_i = (\theta_{i1}, \theta_{i2})$ is starting and ending angle
 3. c_i indicates RGB color
 4. η_i is smoothness of boundary
2. δ is a hyperparameter that controls the stroke of the visualized boundaries
3. The edge with the largest index l is in the front

α clipping:

3. All alpha values thresholded (resulting image has no transparency, all opaque). This results in clean edges where blur is parameterized by η_i

Auxiliary Representations

Signed distance map



(a-b) signed distance maps in the radial and axial directions respectively

(c) the unsigned scaled distance map to the vertex

(d-e) the signed distance map to the starting and ending edges of the bottom wedge

(f) the indicator function of the bottom wedge

Signed distance map $d_{it}(x; \Psi)$, $t = 1, 2$, the signed distance map to the starting ($t = 1$) or ending ($t = 2$) edge of the i th wedge.

Radial and axial signed distance maps (a-b)

Radial: $r_{it}(x; \Psi)$

1. The radial signed distance map means the distance from each pixel to nearest point on the edge ray. So cast a perpendicular ray from the pixel to the edge and that is the radial distance
2. Positive if on the side of the edge normal

Axial: $a_{it}(x; \Psi)$

1. Axial direction is the direction along the edge ray itself, starting from vertex. So the axial distance is the distance along the ray of each pixel.
2. Positive is in front of vertex along ray, negative is behind vertex

Unsigned scaled distance to vertex (c) $v_{it}(x; \Psi)$

$$\sqrt{(r_{it}(x; \Psi))^2 + w^2(a_{it}(x; \Psi))^2}$$

Where w is the scale factor

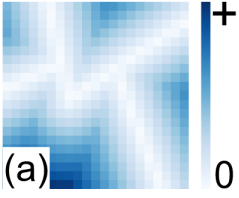
Signed distance map of the i th wedge $d_i(x; \Psi)$ is:

$$d_i(x; \Psi) = \min(|d_{i1}(x; \Psi)|, |d_{i2}(x; \Psi)|) \chi_i(x; \Psi)$$

where $\chi_i(x; \Psi)$ is the indicator function indicating whether pixel x is inside the i th wedge.

1. Distance is negative if the pixel is not inside the given wedge

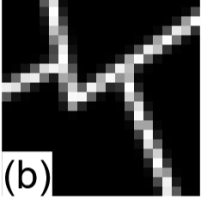
$$\chi_i(x; \Psi) = \begin{cases} 1 & \text{if } \theta_{i1} \leq \arctan2(x - p_i) \leq \theta_{i2} \\ -1 & \text{otherwise.} \end{cases}$$



Unsigned distance map to nearest unoccluded boundary center for each pixel:

$$u(x; \Psi) = \min(|d_1(x; \Psi)|, \dots, |d_l(x; \Psi)|)$$

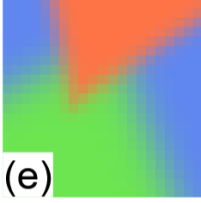
The unsigned distance map measures the distance from each pixel to the nearest boundary.



Boundary center map highlights the center of each unoccluded boundary in patch.

$$b(x; \Psi, \delta) = \exp \left[-\frac{(u(x; \Psi))^2}{\delta^2} \right]$$

Where δ is boundary stroke



Color map is the rendering of the stacked, colored wedges with boundary smoothness and occlusion.

$$c(x; \Psi) = \sum_{i=0}^l c_i \alpha_{l \rightarrow i}(x; \Psi)$$

Computed with alpha clipping, where

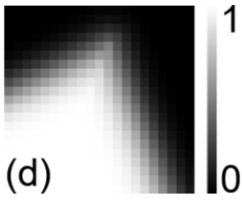
$$\alpha_{l \rightarrow i}(x; \Psi) = \alpha_i(x; \Psi) \prod_{j=i+1}^l (1 - \alpha_j(x; \Psi))$$

is the collective α map from the l th to the i th wedge. The collective α map blends the color at each pixel to accurately render the stacked wedges.

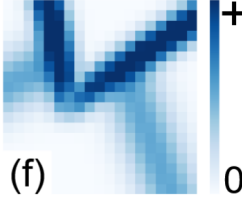
$$\alpha_i(x; \Psi) = \frac{1}{2} \left[1 + \operatorname{erf} \left(\frac{d_i(x; \Psi)}{\sqrt{2}\eta_i} \right) \right]$$

Where $\operatorname{erf}()$ indicates the Gaussian error function, $d_i(x; \Psi)$ is the signed distance map of the i th wedge.

α map where bottom wedge is on the bottom:



The alpha map is the transparency map, and acts to turn sharp edges into blurry edges given η_i



Magnitude of **color derivative map**

$$c'(x; \Psi) = \sqrt{(c(x; \Psi) * G_x)^2 + (c(x; \Psi) * G_y)^2}$$

Where G_x and G_y are Sobel kernels in x and y directions.

Sobel kernels approximate the gradient of the image.

DFD

Pinhole image $Q(x)$ is approximated by a piecewise 2d step function $\bar{Q}(x)$ convolved with a Gaussian blur kernel $k(x, \xi)$ where the stdev. ξ is varying for each patch.

$$Q(x) = \bar{Q}(x) * k(x, \xi), x \in P$$

The piecewise step function represents the image with hard edges, zero blur, zero gradients, no shadows (essentially alpha clipping of the wedges). Then the blur kernel represents the sharpness of the texture in the patch.

Then the captured image can be expressed as:

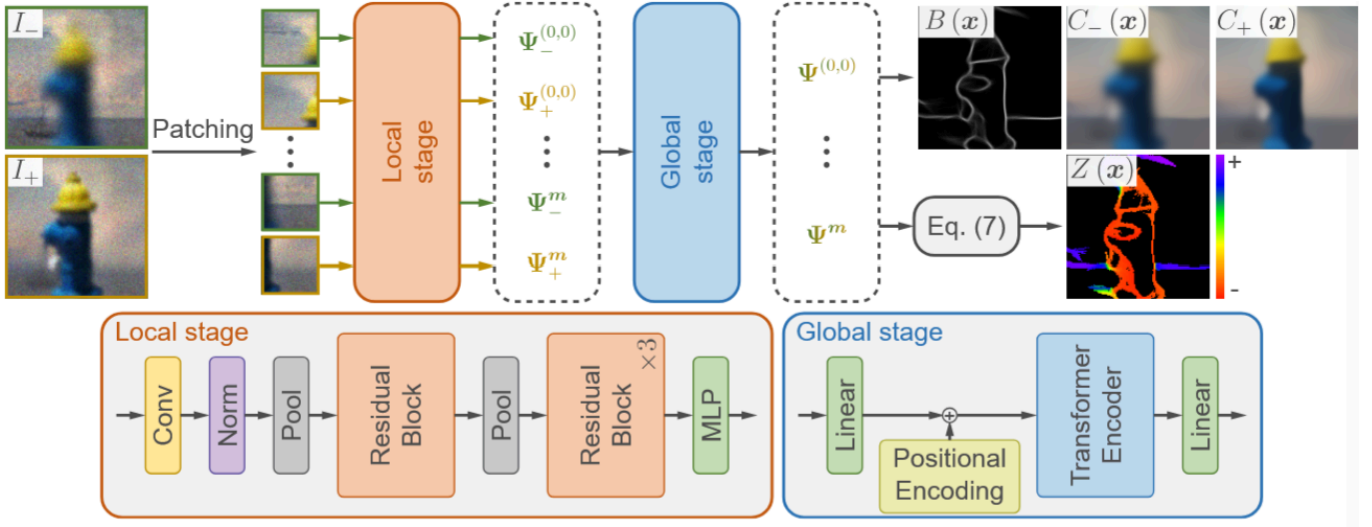
$$I(x) = \bar{Q} * k(x, \sqrt{\sigma(z)^2 + \xi^2})$$

where the $\sqrt{\sigma(z)^2 + \xi^2}$ represents the **boundary smoothness** and is the defocus blur.

At the end we have the depth equation $z(\eta_+, \eta_-)$ which calculates the depth of the boundary given estimated smoothness $\eta_{\pm}$

Note: The two captured images is denoted $I_{\pm}$ with optical power $\rho_{\pm}$

ML Model



Local Stage

Local stage predicts BE repr. per patch independently. $P_{\pm}^m$ denotes patch cropped from one of the two images, $I_{\pm}$, where $m = (m, n)$ denotes center position of patch from original image.

CNN predicts parameters vertex location, angles, boundary smoothness, $\{p_i, \theta_i, \eta_i\}$, then the color info is computed using the parameters using ridge regression.

Global Stage

Transformer Encoder takes in all BE representations $\{\Psi_{\pm}^m, \forall m\}$ and refines them with consistency constraints.

Positional Encoding

Tells each patch where it is in the image:

1. Sinusoidal encoding apparently encodes relative position very well - consistent distance between patches
1. Also normalized for -1, 1

Pairwise Regularization

For each pair of patches with the same center position m (one patch from each defocused image), the global stage outputs a regularized representation:

$$\Psi^m = (\Omega^m, \eta_+^m, \eta_-^m) :$$

$$\Omega^m = \{p_i, \theta_i, c_i, c_0, i = 1, \dots, l\}$$

$$\eta_+^m = \{\eta_{i,+}, i = 1, \dots, l\}$$

$$\eta_-^m = \{\eta_{i,-}, i = 1, \dots, l\}$$

$$\Omega^m:$$

The regularized Blurry Edges repr. includes the boundary smoothness for each of the two patches.

Enforces **defocus consistency**, the pair of patches have the same wedge color and positions, but different boundary smoothness.

Note: c_0 is the "RGB color of the background"

Then use DFD eqn. to calculate depth of each wedge from the boundary smoothness values

Neighbor Consistency

Also trained to promote consistency among neighboring patches in terms of boundary center maps, color maps, color derivative maps.

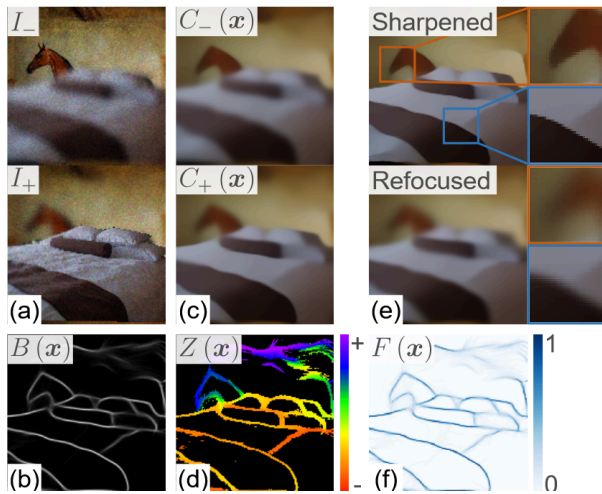
Global Maps

The model calculates global boundary center map, global color map, and global depth image by aggregating all patchwise representations.

Global boundary center and color maps calculated by averaging all per patch maps.

The global sparse depth map $Z(x)$ represents depth values along the boundaries.

Also outputs global confidence map based on boundary center maps



Loss Fn

Local Loss

$$\mathcal{L}_{\text{local}} = \sum_{i=1}^3 \beta_i \mathbb{E}_m(l_i)$$

$\mathbb{E}_m$ denotes the expectation across all patches - meaning that it averages the specific loss l_i across all patches in the image.

Then the local stage total loss is the sum of all average losses.

Individual Losses

l_i is the loss for a single patch.

1. $l_1 = \|c(x; \Psi_{\pm}^m) - P_{\text{clean}, \pm}^m(x)\|^2$ (color error loss ($i = 1$))

1. Predicted color map - noiseless image patch

2. $l_2 = \|c'(x; \Psi_{\pm}^m) - P_{\text{clean}, \pm}^{m, '}(x)\|^2$ (smoothness error loss ($i = 2$))

1. Predicted color map derivative - noiseless image patch derivative from Sobel filtering

3. $l_3 = b(x; \Psi_{\pm}^m, \delta) \cdot u^m(x)$ (boundary localization error loss ($i = 3$))

1. Predicted boundary center map multiplied with true unsigned distance map

2. Unsigned distance map is 0 on boundaries, while boundary center map is 1 on boundaries

3. So this loss function penalizes when boundary is mismatched, and the penalty is proportional to the distance the predicted boundary is off by

Beta Updating

All but the color loss have a beta.

Beta increases linearly from 0 to user set max beta vals. The max beta vals are reached when the epoch count reaches a specific "dynamic epoch" set by the user.

Dynamically updating beta helps convergence. This makes sense its first important to get the wedges in about the right position with the color map, then later refine the boundary locations and color derivative map

Local Loss Implementation

Train / Val

```
loss = criteria(est, img_gt, img_gt, bndry_dist, deri)
```

1. Loss is a function of estimated params, g.t. image (l_1), g.t. color derivative map (l_2), g.t. boundary distance map (l_3)

l_1 - we have predicted map and noiseless image

l_2 - we can obtain g.t. derivative color map by Sobel filtering noiseless image

l_3 - calculate unsigned distance map since you know the ground truth shape stack information

How g.t. unsigned distance map is generated given g.t. shape info

1. `boundary_loc` - becomes the boundary center map with stroke 1 pixel

1. for each object in scene:

1. `mask_bndry = mask_blank.copy()` start blank
2. draw contour of object onto `mask_bndry` with stroke 1 color white
3. `shape_ind = np.where(mask > 0)` `mask` is a blank image with the filled white object pasted on it. Then `np.where` finds all the pixels covered by solid object
4. `boundary_loc[shape_ind] = mask_bndry[shape_ind]`
 1. Then, using the output of `np.where`, we replace whatever is in those pixels with the shape outline. This effectively handles the overlapping of boundaries.
2. From boundary map, directly calculate unsigned distance map

Test

Compares accuracy of depth map, no boundary distance map calculation for test set.

Global Stage Loss

Loss function g_i of seven terms:

1. Color error

1. $\|c(x; \{\Omega^m, \eta^m\}) - P_{\text{clean}}^m(x)\|^2$
2. P is the ground truth pair of images, so each color map is paired with the corresponding image patch
2. **Color consistency**
3. $\|c(x; \{\Omega^m, \eta^m\}) - C^m(x)\|^2$
4. Note that $C(x)$ is the global color map computed by averaging all of the local color maps centered at the same position
 1. So only averaging 2 images since $l = 2$
5. So this measures the distance of the particular color map with the average map. The intention is to enforce color consistency

2. Boundary consistency

1. $\|b(x; \Omega^m, \delta) - B^m(x)\|^2$
2. Note $B^m(x)$ is the global boundary map, avg of all boundary center maps centered at m
3. Boundary consistency between the two defocused images

3. Smoothness error

1. $\|c'(x; \{\Omega^m, \eta^m\}) - P_{\text{clean}}^{m,'}(x)\|^2$
2. Color derivative map diff

4. Smoothness consistency

1. $\|c'(x; \{\Omega^m, \eta^m\}) - C^{m,'}(x)\|^2$
2. Diff between each of the pair of color derivative maps and the global average map for consistency

5. Boundary localization

1. $\|b(x; \Omega^m, \delta) \cdot u^m(x)\|^2$
2. Same boundary localization as local but for pairs

6. Depth error

1. Estimated depth is thresholded in the loss function by the boundary center map (this is unlike FocalSplit where depth is not thresholded)
2. $\|\sum_{i=1}^l H(b_i(x; \Omega^m, \delta) - \tau) \cdot z_i^m - Z^{*,m}(x)\|^2$
3. Where H is the heaviside step fn that is 1 when argument is positive and 0 when negative
 1. τ is the threshold parameter

Coefficients

9. Authors find that dynamically changing β_i and γ_i during training helps with convergence.

Training Dataset

Synthetic data of only basic geometries with random depth value from 0.75m to 1.18m. Images all have applied Poisson-Gaussian noise.

Test Set

Cant use RGBD datasets since to render a defocused image you require occluded background information. (the color of the blurry boundary of a foreground object requires the color of background directly behind it)

Foreground and background are selected from two datasets.

Foreground images from the MS-COCO dataset, which includes segmentation masks so they can extract foreground objects.

Background images from the Painting dataset.

Both foreground and background can have varying depth values

1. Create defocused images by applying PSFs given created depth map

Patch Size

Using patch size 21×21 with stride 2